

Compute Express Link (CXL): Enabling Heterogeneous Data-Centric Computing With Heterogeneous Memory Hierarchy

Debendra Das Sharma , Intel Corporation, Santa Clara, CA, 95052, USA

Compute Express Link is an open industry standard interconnect offering caching and memory semantics on top of peripheral component interconnect-Express, with resource pooling and fabric capabilities. In addition to providing high-bandwidth and low-latency connectivity between host processor and accelerators, smart network interface cards, and memory expansion devices, it also enables resource pooling across multiple systems for scalable, power-efficient, and cost-effective computing. This article delves into the microarchitectural design to deliver power-efficient performance based on our experience designing a Xeon central processing unit and field-programmable gate array with this technology with demonstrated silicon interoperability.

Compute Express Link (CXL) is an open industry standard interconnect offering high-bandwidth, low-latency connectivity between host processing entities, heterogeneous memory, and smart input/output (I/O) devices, running on the Peripheral Component Interconnect-express (PCIe) 5.0 and 6.0 physical layer (PHY) infrastructure, as shown in Figure 1(a).⁹

CXL supports dynamic multiplexing of I/O (CXL.io), coherency (CXL.cache), and memory (CXL.memory) protocols [see Figure 1(b)]. CXL maintains a unified, coherent memory space between the central processing unit(s) (CPU) (host processor) and memory attached to CXL device(s). This allows both the CPU and CXL device to share memory resources coherently for higher performance, as shown in Figure 1(a). CXL is an asymmetric protocol. The CPU is primarily responsible for coherency management through its existing home agent [see Figure 1(b)]. This asymmetry helps reduce device cost and complexity, leading to wider adoption.

CXL 1.0/1.1 defines three types of devices.^{1,3} Type-1 devices are accelerators (e.g., smart network interface cards) that use the coherency semantics along with PCIe-style direct memory access (DMA) transfers. Type-2 devices are accelerators [e.g., general purpose graphics processing unit and field-programmable gate array (FPGA)] with local memory that can be mapped

in part to the cacheable system memory. Type-3 devices are used for memory bandwidth and capacity expansion, and can be used to connect to different memory types. The low-latency, power-efficient, high data rates of PCIe PHY offers a cost, power, and pin-efficient alternative to adding more double data rate memory (DDR) channels to the server CPUs, while offering flexibility in system topologies due to longer trace lengths, which results in alleviating the power delivery as well as cooling constraints.⁹

CXL 2.0 provides support for switching, Type-1/2 device pooling, Type-3 memory pooling at finer granularity among independent hosts, managed hot plug for reassigning pooled resources as well as replacing devices, persistence flows, and fabric manager for managing resources, while being fully backward compatible with CXL 1.1/1.0. Thus, CXL 2.0 provides a mechanism for CXL to extend to be a rack-level, low-latency interconnect with load-store semantics extending from the single system level in CXL 1.1. This enables dynamically composable system at the rack level.⁹ CXL 3.0 doubles the data rate while adding capability to construct large systems with fabric constructs.^{6,7} CXL 3.0 is backward compatible with CXL 2.0 and CXL 1.0/1.1.

This article extends our prior work covering protocol and implementation aspects of CXL 1.0 and CXL 2.0⁹ to double the bandwidth with 64.0-GT/s pulse amplitude modulation 4-level (PAM-4) signaling. Our proposed flit formats for 64.0 GT/s in Sharma's work^{10,11} that has been adopted in PCIe 6.0⁸ and CXL 3.0⁶ specifications. CXL protocol basics along with our proposed slot placement to map CXL

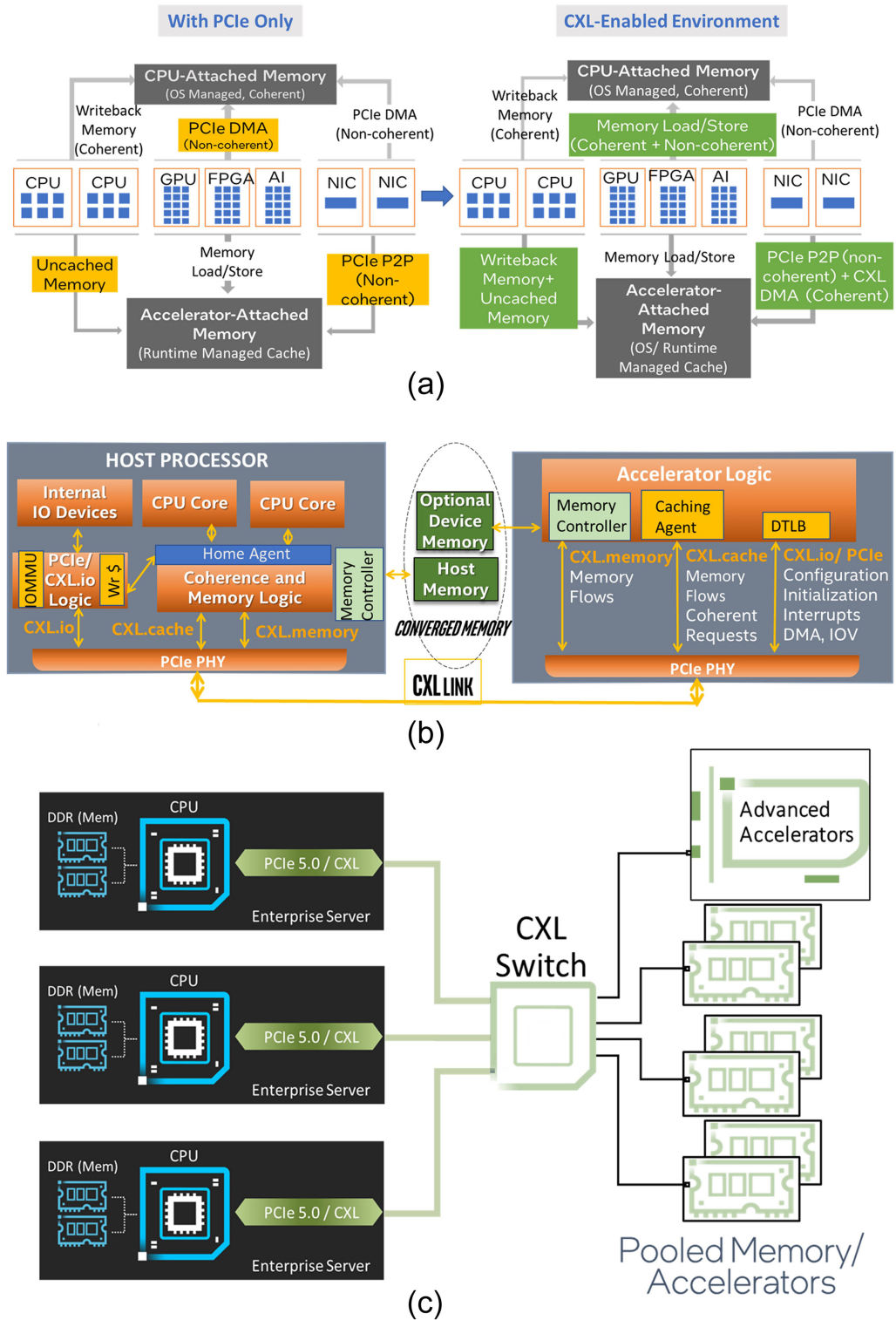


FIGURE 1. Usage models enabled by CXL. (a) CXL enables coherency and memory semantic on top of PCIe. (b) Dynamic multiplexing of three protocols with CXL on PCIe PHY. (c) Resource pooling with CXL 2.0.

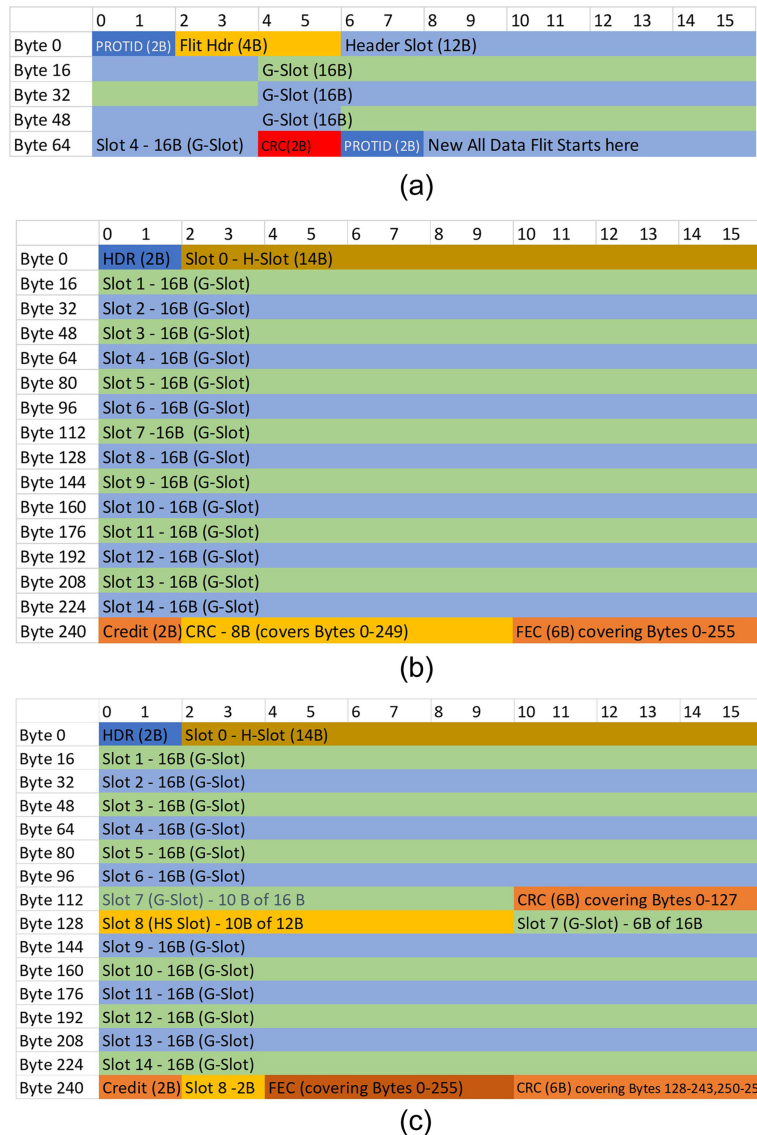


FIGURE 2. Flit layout for various flit types. (a) 68-byte flit layout: 2-byte ProtID + 64-byte payload + 2-byte CRC. (b) 256-byte standard flit layout. (c) 128-byte LO flit layout.

protocol to these new flits at 64.0 GT/s appears in the “CXL Interconnect Architecture” section. Our proposed microarchitectural approach and the resulting performance details are discussed in the “CXL Implementation” section. Finally, the “Conclusions” section concludes this article.

CXL INTERCONNECT ARCHITECTURE

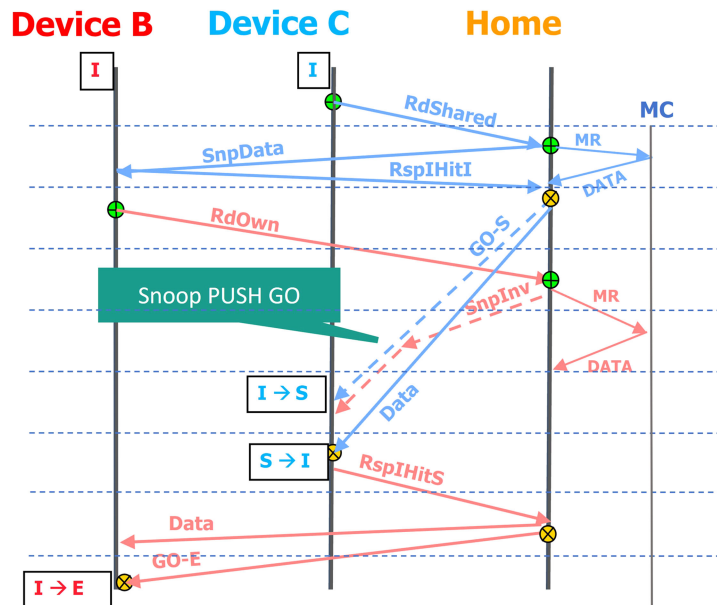
Plug and Play Between CXL and PCIe

CXL runs on PCIe PHY and supports x16, x8, and x4 link widths at 32.0 and 64.0 GT/s. It also supports 16.0-

and 8.0-GT/s data rates and x1/x2 widths in degraded mode.^{1,2} A PCIe slot can be used for either a CXL device or a PCIe device. This offers a customer the flexibility of choosing the mix of PCIe and CXL devices in the platform, depending on the usage model.⁹

Dynamic Multiprotocol Support in CXL

The CXL.io protocol is based on PCIe and is used for the functions, such as device discovery, configuration, initialization, I/O virtualization, and DMA using noncoherent load-store semantics, which follows the producer-consumer ordering model for data consistency. CXL.io



(Here two CXL devices B and C request the same cache line, for exclusive and shared ownership respectively. The home agent receives C's request first and provides data and shared ownership (GO-S). However, a subsequent snoop arising due to B's access (Snplnv) has been sent to C while the GO-S is still in flight. The ordering rules require that the snoop be ordered behind the GO-S since it is to the same cache line, resulting in C getting the data S, possibly consuming it, prior to invalidating the line (RsplHitS). Without this ordering C would have provided a Rspl if the Snplnv arrived before the GO-S and the same cache line would have been in E state in B and S state in C, resulting in a MESI protocol violation)

FIGURE 3. Conflict resolution example with snoop-GO ordering.

uses the same transaction layer packets (TLPs) and data link layer packets (DLLPs) as PCIe.^{1,2,3,4} The packet-level cyclic redundancy check (CRC) of TLP/DLLP is present in the 68-byte flit mode only. In the other two flit modes, the flit CRC is used and the TLP/DLLP does not have their individual CRC.

CXL.cache enables a device to cache data from the host memory, employing a simple request and response protocol.

CXL.memory allows a host processor or other CXL devices to access memory attached to a CXL device. Requests are at 64-byte cache line-sized granularity.

CXL is an asymmetric protocol. It deploys hardware managed modified, exclusive, shared, invalid (MESI) coherency protocol for ensuring data consistency in CXL.cache and CXL.memory protocols. Only host physical addresses are used for CXL.cache and CXL.memory protocols. The home agent in the host processor is responsible for orchestrating cache coherency and resolving conflicts across multiple caching agents (CXL devices, local cores, other CPU sockets, etc.) while

mapping the CXL protocol to its internal cache coherency mechanism. The device with a caching agent implements a simple set of commands (e.g., requests and snoops). Similarly, device memory mapped to the system memory is also managed by the home agent. Device caches are tracked by the host processor through a snoop filter to ensure that the devices do not receive unnecessary snoops. This enables simpler implementation of the caching agent on the device side by leveraging the existing home agent functionality in the host processor(s).⁹ The home agent and snoop filter in modern host processors are scalable enough to track hundreds of cores. The host processor is expected to size its tracking data structures for number of CXL devices/ ports it supports.

Flow-Control Unit (Flit) for Three Generations of CXL

CXL multiplexes the three protocols on PCIe PHY layer. The unit of transfer is flit based. CXL 1.0/1.1/2.0 flit size is 68 bytes; comprising a 2-byte protocol-ID, a 64-byte

TABLE 1. CXL.io realizable b/w on a x16 32-GT/s link or a x8 64.0-GT/s link.

Payload Size (DW)	68-Byte Flit			256-Byte Standard Flit			128B Latency-Optimized Flit		
	100% Read	100% Write	50-50 RW	100% Read	100% Write	50-50 RW	100% Read	100% Write	50-50 RW
1	9.8	8.4	9.1	14.7	11.8	13.1	14.5	11.6	12.9
4	26.2	23.5	29.4	33.6	29.4	39.2	33.1	28.9	38.6
8	36.2	33.6	47.1	42.8	39.2	58.8	42.1	38.6	57.8
16	44.9	42.8	67.3	49.6	47.1	78.5	48.7	46.3	77.1
24	48.7	47.1	78.5	52.3	50.4	88.3	51.4	49.6	86.8
32	50.9	49.6	85.6	53.8	52.3	94.1	52.9	51.4	92.6
64	54.6	53.8	99.2	56.2	55.4	104.6	55.3	54.4	102.8
128	56.7	56.2	107.7	57.5	57.1	110.8	56.5	56.1	108.9
256	57.7	57.5	112.5	58.2	57.9	114.1	57.2	57.0	112.2
512	58.3	58.2	115.0	58.5	58.4	115.9	57.5	57.4	113.9
1024	58.6	58.5	116.4	58.7	58.6	116.8	57.7	57.6	114.8

(100% read is outbound data flow only; 100% write is inbound data flow only; 50–50 RW is equally distributed between the two directions. As expected, the bandwidth reaches the raw bandwidth of 64 GB/s/direction as the payload size increases due to amortization of the same overhead over a larger payload. Also, the 256B and 128B LO flits get better bandwidth efficiency since they do not have the framing and per-TLP CRC overhead).

CXL 3.0 adopted our proposed flit and slot arrangement: 256-byte flit [see Figure 2(b)] and 128-byte latency-optimized (LO) flit [see Figure 2(c)] with 0-latency adder over CXL 2.0. The G-slots are 16 bytes and can be used for header(s) and data whereas the H/HS-slots (14/12 bytes) are for header(s) only.

The type of flit to be used is negotiated initially and does not change till the link is reset (i.e., goes to detect state). The 68-byte flit is only used if the highest common negotiated data rate is less than 64.0 GT/s.

CXL.cache Protocol

There are three message classes, each with its independent flow-control/ credit: Request (Req), response (Rsp), and data. The data carries the cache line data payload for Req or Rsp transactions.

In the device to host direction (D2H), CXL.cache Req includes read requests (e.g., Rd_shared for shared ownership, RdOwn for exclusive access) and write requests. The corresponding H2D response is global observability (GO). The H2D Req includes snoops (e.g., SnpInv, to invalidate the cache line being snooped; SnpCur, which directs the device to take a snapshot, if needed) and the corresponding D2H response (e.g., Rspl, which indicates the device moved to I state, RspSHitSE, which indicates the device moved from E state to S state as a result of the snoop). CXL.Cache is unordered within and across message classes with

the exception that a snoop must push GO to the same cache line to enforce MESI protocol, which can occur due to conflicts, as demonstrated in Figure 3. Conflicts occur when multiple caches want access to the same cache line simultaneously.

CXL.mem Protocol

CXL.mem is a transactional and directional protocol between the host processor (M) and CXL device (S). It supports a 2-bit metadata to track the directory state. Only requests are sent from the host to the device. Each request is sent in one of two channels: M2SReq (e.g., read request) and M2S RwD (e.g., writes). The device to host-side transactions are only responses, sent on two channels: S2M no data response (NDR) and S2M response with data (DRS).

CXL IMPLEMENTATION

CXL 1.0/1.1 is implemented in Intel's Sapphire Rapids (SPR) CPU. The SPR silicon has been extensively tested with Intel's FPGA on a x16 CXL link at 32.0 GT/s, running trillions of transactions per day on each link. Several companies have announced interoperability of their devices with SPR silicon.^{12,13,14,15} We have also been able to validate the latency and bandwidth of our CXL 1.0/1.1 implementation against the calculated numbers with 68-byte flit, as described in this section.

TABLE 2. Realizable bandwidth with CXL.mem for different traffic mixes across three flit types.

	Type-3	Type-2
M2S Transactions	$(x+y)$ Req + y 64B Cache Line Data	
S2M Transactions	x DRS, y NDR, x 64B Cache Line Data	x DRS, $x+y$ NDR, x 64B Cache Line Data
M2S Slots (all Flit types)	$x + 5y$ (Each Req is 1 slot. For 256 Byte or 128B LO Flit: $x+y$ any Slots for Req + $4y$ G-slots for data; $\frac{4y}{x+5y} < \frac{13}{15}$ or $\frac{14}{15} \Rightarrow$ the total slots is $x + 5y$)	
S2M Slots (68-Byte Flit)	$\frac{x+y}{2} + 4x$	$\frac{y}{2} + 5x$
	Note: 1 slot: 1 DRS + 1 NDR or 2 DRS or 2 NDR	
S2M Slots (256-Byte Flit) ¹	$4x + \left(\frac{4x}{14}\right) + \left(\frac{(x+y)-\left(\frac{4x}{7}\right)}{3}\right)$	$4x + \left(\frac{4x}{14}\right) + \left(\frac{(2x+y)-\left(\frac{4x}{7}\right)}{3}\right)$
S2M Slots (128B LO Flit) ¹	If $c1$: $\frac{4x}{13} * 15$ Else: $4x + \frac{2*4x}{13} + \frac{(x+y)-4*\left(\frac{4x}{13}\right)}{3}$	$4x + \frac{2*4x}{13} + \frac{(2x+y)-4*\left(\frac{4x}{13}\right)}{3}$
max_slot	max (M2S Slots, S2M Slots)	
M2S Efficiency B/W	$Link_{Efficiency} * \frac{4y}{max_slot}$	$Link_{Efficiency} * \frac{4y}{max_slot}$
S2M Efficiency B/W	$Link_{Efficiency} * \frac{4x}{max_slot}$	$Link_{Efficiency} * \frac{4x}{max_slot}$

¹ $c1$ is True if we have all G-slots with data than can be covered by the H/ HS-slots. For 256-Byte standard Flit: it has 1-H slot and 15 G-Slots in a Flit – an H-Slot can cover at most 2 cache lines which is 8 G-slots, which is < 15 G-slots – so $c1$ is always False. For 128-Byte Latency-optimized Flit for Type-2, each cache line needs at least one DRS and one NDR – so the two H/HS-slots can cover at most two cache lines with 8 G-slots which is less than the 13 available G-slots – so $c1$ is always False. For 128-Byte Optimized Type-3: $c1$ is True if $4x$ (data slots) $> \frac{13}{4} * (x+y) \Rightarrow x > \frac{13}{3}y$. Note that “128B LO” means “128 Byte Latency-Optimized” Flit.

(a) CXL.Mem Realizable Bandwidth across all three types of Flits

Data B/W (GB/s)		Type-3		Type-2	
x16 @ 32G or x8 @ 64 G (Raw B/W: 64 GB/s/dir)		M2S	S2M	M2S	S2M
1R/0W	68B Flit	0	53.5	0	48.1
	256B Flit	0	54.0	0	50.3
	128B LO	0	51.9	0	49.1
1R/1W	68B Flit	40.1	40.1	40.1	40.1
	256B Flit	39.9	39.9	39.9	39.9
	128B LO	39.9	39.9	39.9	39.9
2R/1W	68B Flit	25.3	50.7	22.9	45.8
	256B Flit	26.0	52.1	24.3	48.6
	128B LO	25.4	50.9	24.3	47.5

(b) CXL.Mem realizable bandwidth for different traffic mixes across 3 Flit Types

CXL Microarchitecture

Figure 4(a) is a representative microarchitecture of host processor, such as SPR. The memory size, caching

hierarchy, and cache sizes are meant to provide a scale and not the exact magnitude. The last level cache (LLC) covers the lower levels of cache in the hierarchy

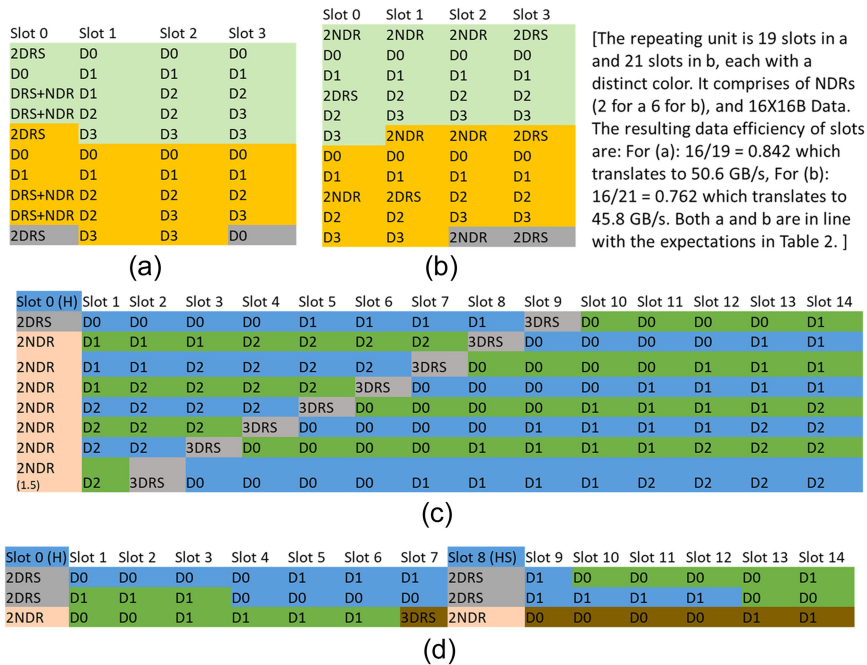


FIGURE 5. Placement of data/HDR in the slots for 2R/1W in CXL.Mem. (a) Type-3 with 68-byte flit format. (b) Type-2 with 68-byte flit format. (c) 256-byte standard flit placement—23 DRS cache lines and 12 NDR (vs 11.5 to be perfect repeat pattern)—Slot 0 is an H slot; the rest are g-slots. (d) 128-byte optimized flit with placement. Slot 0 is an H-slot; Slot 8 is an HS-slot; the rest are G-slots. This will continue for a long time before repeating.

as well as the write caches associated with the PCIe/CXL.io stack inside the CPU. The LLC also covers caches across all CXL devices. A snoop filter provides the home agent with information on when to snoop a given CXL device. Memory, whether locally connected to the CPU (e.g., DDR), or belonging to the CXL device mapped to the system address space is under the purview of the home agent [represented by blue and green colors in Figure 4(a)].

Figure 4(b) represents the microarchitectural block diagram of our CXL IP. Our IP supports industry standard interfaces, such as PHY interface for PCI express (PIPE)¹⁶ and logical PHY interface (LPIF).¹⁷ We have implemented a prespecification version of the CPI¹⁸ and SFI¹⁹ interfaces.

The PHY block has the analog circuitry and is responsible for serializing/deserializing the data. It connects to the PHY logical block through the parallel interface of PIPE (32 bits at 1 GHz per lane for a 32.0-GT/s Link). The PHY logical block is responsible for link training, equalization, (de)scrambling, precoding, lane reversal, link width degradation, polarity inversion, clock domain crossing, clock compensation, deskew, and PHY framing.

The LPIF interface connects our PHY logical to the Arb/Mux block and from there to the multiple link layers. The Arb/Mux block performs the arbitration

and multiplexing between the two stacks (CXL.io and CXL.cache + CXL.mem, collectively known as CXL.\$ Mem) and also virtualizes the PHY to the independent link layer stacks for power management.^{1,3,9}

The link layer performs CRC checks, manages credits, and handles link level retry whereas the transaction layer performs the flit packing/ unpacking, stores transactions in the appropriate queues, and processes the transactions.⁹ It should be noted that in future CXL 3.0 implementations, when 256- or 128-byte LO flit is negotiated, the logical PHY will perform the CRC and replay functionality, which is common across all stacks and the link layer with each stack will continue to provide the same functionality only for the 68-byte flit mode for that stack for backward compatibility. The CXL.io/PCIe transaction layer queues the transactions, processes them, and enforces producer-consumer ordering.

CXL.cache and CXL.mem Pipeline With Latency

Figure 4(b) details the CXL.\$Mem path, including latency. The latency includes the transmit plus the receive path. We have implemented several latency optimization techniques over a standard PCIe PHY

logical implementation. These include bypassing the 128-/130-byte encoding (if negotiated during link training³), bypassing the logic, and serializing flops needed to support degraded mode (if one or more lanes are bad or if narrower width is invoked for power management purposes), bypassing the deskew buffer if the lane to lane skew is less than half the internal PHY logical clock period, adopting a predictive policy of processing entries from the elastic buffer (versus waiting for the clock domain synchronization handshake for every entry), etc. The PHY and PHY logical has a latency range of 15–19 ns depending on whether a common reference clock or independent reference clocks are deployed. If the components are within a single chassis, we expect a common reference clock with a 15-ns latency. However, if the link connects across chassis, we expect independent reference clocks, with a 4-ns latency penalty due to clock crossing as well as sync header (HDR) overheads.

The link layer and transaction layer paths have a low latency since they are natively flit based. CXL.\$Mem protocol muxing at the PHY level (versus higher level of the stack) helps deliver a low latency path for CXL.\$Mem traffic. This eliminates the higher latency in the PCIe/ CXL.io path due to the support for variable packet size, ordering rules, access rights checks, etc., conforming to the fundamental guiding principle of CXL specifications.^{1,3}

The total latency from the SERDES pin to the internal application layer (CPI or equivalent interface) and back is 21 ns in common reference clock mode and 25 ns in independent reference clock mode. One IP vendor has reported a round-trip latency of 25 ns,¹⁵ presumably in the common clock mode. Minor variations in this number are expected due to factors, such as propagation delays due to placement of the various blocks, process technology, and the type of PHY design. For example, an analog to digital converter-based PHY would likely have an additional latency of about 5 ns over a decision feedback equalizer/continuous time linear equalizer-based receiver design. Our measured numbers gives us confidence in the CXL specification pin-to-pin latency targets of 80 ns for a CXL.Mem access targeting dynamic random access memory/high bandwidth memory or a 50-ns response for a snoop.^{3,4,6}

With a 21/25-ns round-trip latency, even after allowing an additional 10 ns for variation due to different design practices, a very conservative 10-ns flight time with retimers, we expect a latency adder of 70 ns ($2 \times 25 + 10 + 10$) for a memory access across CXL Link. With CXL memory, the idle load-to-use latency is expected to be between 150 and 175 ns. This is similar to memory access across a CPU–CPU cache-coherent link and well within the latency operating points of CPU.²⁰

For CXL.io, as in PCIe, the pin-to-pin round-trip latency for a memory read to local memory to completion in Xeon is less than 275 ns with an LLC miss and IOTLB hit.

Realizable Bandwidth on a x16 CXL Link

A x16 CXL link has a raw bandwidth of 64 (128) GB/s per direction at 32.0 (64.0) GT/s data rate. The link efficiency and the underlying IP implementation for various protocols determine the realizable data bandwidth. Everything except data payload is considered overhead: header, CRC, protocol ID/ flit HDR, etc. We evaluate the realizable data bandwidth for each of the three protocols (CXL.io, CXL.cache, and CXL.mem) with different traffic mixes for each flit type.

With 68-byte flit mode, there are three overheads that are common: 128/130 represents the sync HDR overhead (which can be reclaimed when sync HDR bypass is supported), 374/375 represents the bandwidth loss due to SKP ordered sets for common clock (higher for other mode), and 64/68 representing the flit overhead (2 bytes each for protocol ID and flit CRC). This results in an efficiency of 0.924 with sync HDR on and 0.939 when sync HDR is off. This represents the $\text{Link}_{\text{efficiency}}$ for CXL.cache and CXL.mem (which includes credits and ack/nak).

For CXL.io in 68-byte flit mode, assuming a 2% DLLP overhead, $\text{Link}_{\text{efficiency}}$ can be expressed as 0.906 and 0.92 for the cases where sync HDR is on or off, respectively. Note that since native PCIe does not have the flit mode, the $\text{Link}_{\text{efficiency}}$ is $0.982 \cdot (1 - \text{DLLP}_{\text{overhead}}) = 0.96$, a 6% improvement over CXL.io, assuming the same DLLP overhead of 2%.

For 256- and 128-byte LO flits, for CXL.cache and CXL.mem, the $\text{Link}_{\text{efficiency}}$ is 15/16 ($= 0.938$), since there are 15 slots and one slot equivalent is spent on FEC/ CRC/ HDR, etc. For CXL.io, the unit is double word (DW = 4 bytes), the overhead is 6 bytes (HDR, DLLP) + 6 bytes (FEC) + 8 bytes (for 256 bytes)/12 bytes (for 128-byte LO) for CRC,^{10,11} a total of 20 bytes for 256-byte flit and 24 bytes for 128-byte LO flit; with $\text{Link}_{\text{efficiency}} = (256 - 20 \text{ or } 24) / 256 = 0.92$ for 256 bytes and 0.91 for 128-byte LO flit.

CXL.io: There are three representative traffic mixes to benchmark performance in CXL.io and PCIe: 100% reads, 100% writes, and 50–50 read–write. Let D represent the data payload per TLP in DW. The header is 3-DW for completions and 4-DW for requests (read/write). An additional overhead of 2-DWs for framing and CRC is incurred per TLP with 68-byte flit ($\text{FT_CRC} = 2$), which is not present in the 256- or 128-byte LO flits ($\text{FT_CRC} = 0$). The 100% read case has data flow only in outbound direction, which

has a 3-DW header per completion. The 100% write case has data flow only in inbound direction, which has a 4-DW header per write. The 50/50 R/W cases assumes equal data transferred for read and write. The bandwidth efficiency is equal in each direction for the 50/50 R/W cases where the inbound direction has two 4-DW headers one for read request and one for the write (8 DW for the two headers), which dictates the bandwidth efficiency. Thus, the bandwidth efficiency can be expressed as

$$\begin{aligned} \text{LinkEfficiency} & * \frac{D}{D + 3 + \text{FT_CRC}} \quad (100\% \text{ read}) \\ \text{LinkEfficiency} & * \frac{D}{D + 4 + \text{FT_CRC}} \quad (100\% \text{ write}) \\ \text{LinkEfficiency} & * \frac{D}{D + 8 + 2 * \text{FT_CRC}} \quad (50/50 \text{ R/W}). \end{aligned}$$

Table 1 summarizes the realized CXL.io bandwidth for the various payload sizes for each of the three traffic mixes for each of the three flit types.

CXL.Mem: A memory read from CPU is sent as MemRd in M2S. It results in a response, which comprises of an S2M DRS HDR (MemData) and 4 slots of data (64 bytes) for a Type-3 device. If the device is a Type-2 device, an additional S2M NDR for completion (Cmp) is needed. For the writes, the CPU sends an M2S Req (MemWrite) and four slots of data, generating an S2M NDR as Cmp.

The headers are placed optimally in a slot. The packing permutations of various headers in a slot are represented with H* numbers. For example, in 68-byte flit, H3 enables S2M DRS + S2M NDR whereas H5 packs 2 S2M DRS. The scheduler in our flit packing logic for our CXL 1.0/1.1 implementation follows a greedy algorithm for CXL.Mem by prioritizing H3 for the first 16-byte slot in a flit followed by H5 and populating data in the remaining flits if less than 64 bytes of data are scheduled to be sent. If 64 bytes or higher data are scheduled to be sent, it schedules an all-data flit.

For the 256-byte standard or 128-byte latency-optimized flit, we have additional constraints to consider. For example, we can (and will) use the H/HS slot for header only. The scheduling will ensure that no G-slot will go empty when there is data (or header) to be sent. Thus, headers that precede data will be prioritized and opportunistically placed in H/HS slots while making sure that no more than five cache lines worth of data are to be scheduled. We also ensure that other headers make forward progress.

Table 2(a) provides the bandwidth efficiency by computing the number of slots required in each direction for Type-2 and Type-3 devices for each of the three flit types for all types of traffic mixes represented as x reads and y writes. Table 2(b) represents the realizable data bandwidth for three representative cases for the representative traffic mixes of 1R/0W, 1R/1W, and 2R/1W for Type-2

and Type-3 devices for each of the three flit types for the same raw bandwidth. In general, the 256-byte standard flit has slightly better efficiency followed by 128-byte LO flit over the 68-byte flit, even with FEC and higher CRC overheads, since information, such as protocol ID and CRC, is better amortized in larger flit sizes.

Figure 5(a) and (b) demonstrates the result of a recurring pattern of 2 read + 1 write with a Type-3 and a Type-2 device for 2R/1W our 68-byte flit implementation, with bandwidth very close to the optimal bandwidth projection in Table 2. We present the S2M direction only since that is the upper bound. Figure 5: Placement of data/HDR in the slots for 2R/1W in CXL.Mem and Figure 5(d) represents possible placement with the 256 and 128-byte latency-optimized (LO) flits and can change as we refine the algorithm going forward.

CXL.Cache: We consider a device reading a cache line from memory using RdCurr in D2H resulting in an H2D DATA_HDR (24 bits, 4 of these can be allocated in a slot) plus H2D data. Thus, a x16 CXL device would get $(16/17) * 0.94 * 64 \text{ GB/s} = 56.6 \text{ GB/s}$ of bandwidth with reads from the processor with 68-byte flit format. For 256- and 128-byte latency-optimized flits, these data are the bottleneck since the H slot can have up to 4 H2D DATA_HDR. Hence, the realizable bandwidth are: $(14/16) * 128 = 112 \text{ GB/s}$ for 256-byte flit and $(13/16) * 128 = 104 \text{ GB/s}$ for 128-byte latency-optimized flit.

For writes, a D2H Req (RdOwn) results in an H2D response with data. The device then issues a D2H dirty evict, obtains an H2D Resp (Wr Pull), which causes it to do a D2H Mem Wr. This results in 3 HDRs (D2H Req RdOwn, D2H dirty evict + Data HDR) for every cache line worth of data in the D2H direction. For 68-byte flit, the 3 HDRs occupy two slots (e.g., D2H Req RdOwn + Data HDR, D2H dirty evict) resulting in $(4/6) * 0.94 * 64 \text{ GB/s} = 40 \text{ GB/s}$ per direction. For the 256-byte as well as 128-byte latency-optimized flit formats, that is 4 G slots and 2.25 any slots (one each for the Req and evict dirty and 1/4 for the Data_HDR since we can have 4 of them in any slot) in D2H direction, resulting in a realizable data bandwidth of $(4 \text{ data slots} / 6.5 \text{ total slots}) * (15/16) * 128 = 73.8 \text{ GB/s}$.

CXL SPECIFICATION IS GAINING WIDE TRACTION IN THE INDUSTRY DUE TO THE SIMPLICITY OF IMPLEMENTING LOW-LATENCY CACHING AND MEMORY SEMANTICS ON A WELL-ESTABLISHED PCIe INFRASTRUCTURE.

CONCLUSION

CXL specification is gaining wide traction in the industry due to the simplicity of implementing low-latency caching and memory semantics on a well-established PCIe infrastructure. We are seeing CXL 1.0/1.1 being deployed at the chassis/node level. We expect to see CXL expand across multiple racks and offer high reliability with low-latency load-store access for multiple applications in the near future.

ACKNOWLEDGMENTS

We would like to thank Rob Blankenship and Balint Fleischer for their help with this article.

REFERENCES

1. D. D. Sharma, "Compute Express Link," White Paper, Compute Express Link Consortium, Mar. 2019. [Online]. Available: https://docs.wixstatic.com/ugd/0c1418_d9878707bbb7427786b70c3c91d5fbd1.pdf
2. PCI-SIG, "PCI Express Base Specification Revision 5.0, version 1.0," May 2019. [Online]. Available: <https://pcisig.com/specifications>
3. CXL Consortium, "Compute Express Link 1.1 specification," Jul. 2019. [Online]. Available: <https://www.computeexpresslink.org/download-the-specification>
4. CXL Consortium, "Compute Express Link 2.0 specification," Sep. 2020. [Online]. Available: <https://www.computeexpresslink.org/download-the-specification>
5. D. D. Sharma and S. Tavallaei, "Compute Express Link™ 2.0," White Paper, CXL Consortium, Nov. 2020. [Online]. Available: https://b373eaf2-67af-4a29-b28c-3aae9e644f30.filesusr.com/ugd/0c1418_14c5283e7f3e40f9b2955c7d0f60bebe.pdf
6. CXL Consortium, "Compute Express Link 3.0 specification," Aug. 2022. [Online]. Available: <https://www.computeexpresslink.org/download-the-specification>
7. D. D. Sharma and I. Agarwal, "Compute Express Link 3.0," White Paper, CXL Consortium, Aug. 2022. [Online]. Available: https://www.computeexpresslink.org/_files/ugd/0c1418_a8713008916044ae9604405d10a7773b.pdf
8. PCI-SIG, "PCI Express Base Specification Revision 6.0, version 1.0," Jan. 2022. [Online]. Available: <https://pcisig.com/specifications>
9. D. D. Sharma, "Compute Express Link: An open-industry standard enabling heterogeneous data-centric computing," in *Proc. IEEE Symp. High-Perform. Interconnects*, 2022, pp. 5–12.
10. D. D. Sharma, "A low-latency and low-power approach for coherency and memory protocols on PCI Express 6.0 PHY at 64.0 GT/s with PAM-4 signaling," *IEEE Micro*, vol. 42, no. 20, pp. 37–43, Mar./Apr. 2022.
11. D. D. Sharma, "PCI Express 6.0 specification: A low-latency, high-bandwidth, high-reliability, and cost-effective interconnect with 64.0 GT/s PAM-4 signaling," *IEEE Micro*, vol. 41, no. 1, pp. 23–29, Jan./Feb. 2021.
12. Synopsys Inc, Oct. 2020. [Online]. Available: <https://news.synopsys.com/2020-10-13-Synopsys-Demonstrates-Industrys-First-PCI-Express-5-0-IP-Interoperability-with-Intels-Future-Xeon-Scalable-Processor>
13. Synopsys Inc, "Compute Express Link (CXL)." [Online]. Available: <https://www.synopsys.com/verification/verification-ip/subsystems/compute-express-link.html>
14. Mobiveil, Dec. 2020. [Online]. Available: <https://www.globenewswire.com/news-release/2020/12/02/2138545/0/en/Mobiveil-Announces-Compute-Express-Link-CXL-2-0-Design-IP-Successful-Completion-of-CXL-1-1-Validation-with-Intel-s-CXL-Host-Platform.html>
15. PLDA, Jul. 2020. [Online]. Available: <https://www.brighttalk.com/webcast/18357/420129>
16. Intel Corporation, "PHY interface for the PCI Express, SATA, USB 3.1, display port, and converged I/O architectures," Version 5.1, 2018. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/phy-interface-pci-express-sata-usb30-architectures-3.1.pdf>
17. Intel Corporation, "Logical PHY interface (LPIF) specification," Version 1.0, Mar. 2019. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/technical-specifications/logical-phy-interface-specification.pdf>
18. Intel Corporation, "CXL-cache/mem protocol interface (CPI)," Rev 0.7, Mar. 2020. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/technical-specifications/logical-phy-interface-specification.pdf>
19. Intel Corporation, "Streaming fabric interface (SFI)," Rev 0.7, Mar. 2020. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/technical-specifications/logical-phy-interface-specification.pdf>
20. I. Papazian, "New 3rd gen Intel Xeon scalable processor (Codename: Ice lake-SP)," Hot Chips, Jul. 2020. [Online]. Available: https://hc32.hotchips.org/assets/program/conference/day1/HotChips2020_Server_Processors_Intel_Irma_ICX-CPU-final3.pdf

DEBENDRA DAS SHARMA is currently a senior fellow at Intel, Santa Clara, CA, 95052, USA. Sharma received a Ph.D. degree in computer systems engineering from the University of Massachusetts, Amherst, MA, USA. He is a Senior Member of IEEE. Contact him at debendra.das.sharma@intel.com.